

PhantomGrid

Technical Documentation

CBI Hackathon 2026 | Team ZeroIntent (S.No. 8) | IIIT Kottayam

Role	Name
Frontend - Behavioural Signal Capture	Madapati Jyoti Radithya
Backend - ML Engine + FastAPI	Kontheti Sai Akhilesh
Dashboard + Tests + Threat Model + Security (Lead)	Praising Y Harris Ratnam

1. Problem Statement

Account Takeover (ATO) fraud in Public Sector Banks (PSBs) has reached a critical inflection point. In FY2023, Indian PSBs reported over Rs. 7,400 crore in digital fraud losses, the majority occurring after successful login. Attackers today do not break authentication systems; they bypass them using stolen credentials purchased from dark-web marketplaces.

The fundamental failure of current authentication is temporal: systems verify identity once at login, then trust the session unconditionally. An attacker who presents valid credentials is indistinguishable from the legitimate user by any existing control.

Password mandates, OTPs, and device fingerprinting each address one vector in isolation. None answer the core question at the point of a Rs. 5 lakh fund transfer: Is the person currently operating this session the enrolled account holder? PhantomGrid answers that question continuously, silently, and without friction.

2. Proposed Solution

PhantomGrid is a three-layer passive behavioural authentication engine running transparently beneath a banking portal. It observes natural behavioural signatures from how users interact, patterns that are unconscious, stable, and practically impossible to replicate.

Enrollment (Sessions 1-5): PhantomGrid silently collects behavioural baseline vectors across three independent signal domains during normal transactions. No user action required.

Continuous Scoring (Session 6+): Every subsequent transaction is scored against the enrolled baseline in real time. Composite risk score 0-100 determines: ALLOW the transaction, trigger silent OTP re-auth, or BLOCK the session.

3. System Architecture

Three-tier architecture with clear separation:

TIER 1 - Signal Capture (Madapati Jyoti Radithya): capture.js injected into NexaBank portal captures L1/L2/L3 signals and packages them into a single JSON payload per transaction.

TIER 2 - ML Inference Engine (Kontheti Sai Akhilesh): FastAPI backend runs Isolation Forest anomaly scoring (L1, L2) and DTW rhythm comparison (L3), fuses scores via $L1 \times 0.30 + L2 \times 0.40 + L3 \times 0.30$, applies replay detection, updates adaptive baseline, writes hash-chained audit log.

TIER 3 - Analyst Dashboard (Praising Y Harris Ratnam): Polls GET /logs every 2 seconds, renders live risk gauge, per-layer scores, WHY THIS DECISION panel, BLOCK alert with audio, audit chain integrity badge, and baseline maturity indicator.

4. Technology Stack

API Framework: FastAPI + Python 3.12 + Uvicorn ASGI. Auto-validates with Pydantic v2, generates Swagger docs.

ML Anomaly Detection: scikit-learn IsolationForest - unsupervised, no fraud labels required, per-user baselines from 5 enrollment samples.

Rhythm Comparison: dtadistance DTW - elastic sequence alignment handles natural PIN speed variation.

Database: SQLAlchemy ORM + SQLite. Zero-config for PoC, PostgreSQL-compatible schema.

Security: Python hashlib SHA-256 - tamper-evident hash chain on session_logs + payload signature replay defence.

Frontend: Vanilla HTML/CSS/JS, no build step, fully portable.

Testing: pytest + requests, 8/8 integration tests against live backend.

5. Workflow

Enrollment: User submits payment -> capture.js packages signals -> POST /enroll -> vectors appended to user_profiles -> after 5 samples, enrollment complete, system switches to verify mode.

Scoring: User submits payment -> POST /verify -> SHA-256 replay check (duplicate in 5-min window = force BLOCK) -> L1 continuous_if_risk(), L2 continuous_if_risk(), L3 dtw_to_risk(min DTW distance) -> fusion -> ALLOW/OTP/BLOCK decision -> if ALLOW: append to adaptive baseline (window 20) -> compute SHA-256 hash chain row -> write session_logs -> return {scores, decision, replay_detected}.

6. Database Design

Table user_profiles: id (PK), user_id (UNIQUE), layer1_vectors (JSON blob of [decoy_taps, hesitations] pairs), layer2_vectors (JSON blob of [bene_dwell_ms, avg_iki] pairs), pin_vectors (JSON blob of interval vectors). IsolationForest is fit on-the-fly from raw vectors, no model serialization.

Table session_logs: id (PK), session_id (UNIQUE), timestamp, user_id, layer1_score, layer2_score, layer3_score, composite_score, decision (ALLOW|OTP|BLOCK), row_hash (SHA-256 of this row), prev_hash (SHA-256 of preceding row). The row_hash/prev_hash pair forms a cryptographic hash chain. Any record modification breaks the chain at that exact point, detectable via GET /audit/verify.

7. AI / ML Models

Isolation Forest (Layers 1 and 2): Unsupervised anomaly detection. Builds 100 isolation trees by recursively partitioning feature space. Normal points need many splits to isolate (deep paths). Anomalies need few (shallow paths). Trained per-user on individual enrollment samples, not a global population.

Continuous Scoring Innovation (services/scoring.py): Standard IF saturates on 5-sample baselines. We solve this using IF as an anomaly gate (inlier/outlier) plus a continuous normalized-deviation magnitude. Inliers: 0-45. Outliers: 55-100. For baselines under 10 samples: pure deviation scoring (deterministic, monotonic, always correct).

Dynamic Time Warping (Layer 3): Elastic alignment between keystroke sequences before measuring distance. Handles natural speed variation. DTW distance mapped to 0-100 via $\min(100, \text{dist}/180 \times 100)$. Best-match strategy against all enrolled vectors.

Adaptive Learning: ALLOW sessions appended to baseline (sliding window 20). Model drifts with legitimate behavioural change.

8. APIs and External Services

POST /enroll - Store one enrollment sample. POST /verify - Score session, returns {layer1_score, layer2_score, layer3_score, composite_score, decision, replay_detected}. GET /logs - Last 10 session log rows. GET /maturity?user_id=X - Baseline maturity {samples, required:5, mature, confidence}. GET /audit/verify - Walk hash chain {valid, verified, broken_at_session}.

No third-party ML APIs. No cloud inference services. No external data providers. All computation is local.

9. Security Measures

Replay-Attack Defence: SHA-256 payload signature (canonical JSON). Exact duplicates within 5-minute window are forced to BLOCK with replay_detected:true. Demonstrated live via demo_replay.py.

Tamper-Evident Audit Trail: SHA-256 hash chain on session_logs. row_hash = SHA256(prev_hash | session_data). Modification of any record detected by GET /audit/verify. RBI audit trail compliant.

Input Validation: Pydantic v2 enforces schema at API boundary. HTTP 422 on malformed payloads before any ML code executes.

PII Minimisation: PIN digits never persisted. Only timing intervals stored (cannot be reversed to recover PIN).

STRIDE: Spoofing mitigated by per-user IF models. Tampering by hash chain. Repudiation by immutable session_logs. Elevation of Privilege: /enroll must require auth token in production.

10. Scalability Considerations

SQLite to PostgreSQL: One connection string change. Schema is PostgreSQL-compatible. Enables horizontal scaling.

Model Caching: Currently fits IF on every /verify call. Production: serialise with joblib, cache in Redis.

Real-time Dashboard: 2s REST polling replaces with 50ms WebSocket streaming for production.

Data Localisation: Production deployment on aws ap-south-1 (Mumbai) or azure centralindia. RBI data localisation requirement satisfied.

Inference Time: Total under 5ms per session. Does not block payment flow.

11. Assumptions and Limitations

Assumptions: Users enroll on the same device used for live transactions (PIN rhythm is device-specific). At least 5 enrollment samples before enforcement. /enroll called within an authenticated session. capture.js is not tampered with client-side.

Limitations: Cold start - under 5 samples model reliability is reduced, maturity endpoint signals this. Cross-device degradation - PIN rhythm changes across keyboards, mitigation: per-device baseline profiles. SQLite single-writer limitation - requires PostgreSQL migration for high-concurrency production.

12. Future Enhancements

Mobile SDK: Touch pressure, swipe velocity, accelerometer during PIN entry. 80% of PSB digital banking is mobile. Richer signals, lower FRR.

PostgreSQL + Redis: Drop-in swap for production scale. Redis caches serialised per-user IF models.

Federated Anonymised Population Model: Train on aggregate anonymised data across PSBs. Bootstraps cold-start users from population priors. Privacy-preserving.

WebSocket Streaming: Replace 2s polling with 50ms push for near-real-time analyst dashboard.

Regulatory Dashboard: RBI-ready reporting module with per-user risk history, aggregate fraud metrics, and audit chain export.

PhantomGrid - Passive. Invisible. Unbeatable.

Team ZeroIntent | S.No. 8 | CBI Hackathon 2026 | MNNIT Allahabad | IIIT Kottayam